UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

ESCUELA PROFESIONAL DE INGENIERÍA ELECTRÓNICA



UNIVERSIDAD
NACIONAL
DE COLOMBIA
SEDE MANIZALES

INFORME

GUÍA DE LABORATORIO DE SIMULACIÓN CAPITULO 5

Curso: Robótica

Estudiantes:

Cleyson Valerio Yauri Cupara

Joel Americo Lopez Valdes

Manizales – Colombia
2026

1. Creación de directorios y archivos base

```bash
mkdir -p ~/lab5_navegacion
cd ~/lab5_navegacion
nano braitenberg_base.py
```

Figura 1: Directorios y archivos base.

```
joel@UNAL-Robotica: ~/lab5_navegacion
joel@UNAL-Robotica:~/Desktop$ cd ~/lab5_navegacion
joel@UNAL-Robotica:~/lab5_navegacion$ nano braitenberg_base.py
joel@UNAL-Robotica:~/lab5_navegacion$ python3 braitenberg_base.py
```

Figura 2: comandos ejecutados en la terminal de linux

2. Parte A: Navegación Reactiva (Vehículo de Braitenberg)

Descripción y Código Implementado

Se define un campo escalar $f(x, y) = \frac{1000}{(x-x_c)^2+(y-y_c)^2+50}$ con un máximo en $(x_c, y_c) = (60, 90)$. El robot se equipa con dos sensores con un desfasaje lateral $d = 2$.

```
    import numpy as np
import matplotlib.pyplot as plt

# 1. Definici n del Campo Escalar (Objetivo por defecto en xc=60, yc
   =90)
def sensorfield(x, y):
    xc, yc = (60, 90)
    return 1000 / ((x - xc)*2 + (y - yc)*2 + 50)

# 2. Configuraci n de Par metros Generales
dt = 0.1
Ks = 17
Kv = 14

# 3. Definici n del Estado Inicial del Robot
x, y, theta = 10, 10, 0
traj_x, traj_y = [], []

# Encabezado estructurado para la terminal
print("=" * 65)
print(f"{'Iteraci n':<10} | {'s_left':<10} | {'s_right':<10} | {'v (m/s
   )':<10} | {'w (rad/s)':<10}")
print("=" * 65)

# 4. Bucle Principal de Simulaci n Cinem tica (222 Iteraciones)
for i in range(222):
    # Sensores (offset lateral d=2)
    d = 2
```

```
    s_left = sensorfield(x - d*np.sin(theta), y + d*np.cos(theta))
    s_right = sensorfield(x + d*np.sin(theta), y - d*np.cos(theta))

    # Ley de Control
    v = -Kv * (s_left + s_right)
    omega = Ks * (s_right - s_left)

    # Imprimir resumen ordenado cada 20 iteraciones para no saturar la pantalla
    if i % 20 == 0 or i == 221:
        print(f"{i:<10} | {s_left:<10.3f} | {s_right:<10.3f} | {v:<10.3f} | {omega:<10.3f}")

    # Integraci n Cinem tica
    x += v * np.cos(theta) * dt
    y += v * np.sin(theta) * dt
    theta += omega * dt

    traj_x.append(x)
    traj_y.append(y)

print("=" * 65)
print(f"Posici n Final Alcanzada: X = {x:.2f} m, Y = {y:.2f} m")
print("=" * 65)

# 5. Visualizaci n Estructurada y Est tica de Resultados
plt.figure(figsize=(8, 7))

# Trazo de la trayectoria del robot
plt.plot(traj_x, traj_y, 'b-', linewidth=2.5, label="Trayectoria del Robot (N=222)")

# Marcar punto de inicio y punto objetivo
plt.scatter([10], [10], c='green', s=120, zorder=5, label="Inicio (10, 10)")
plt.scatter([60], [90], c='red', s=150, marker='*', zorder=5, label="Objetivo (60, 90)")

# Formato visual
plt.title("Navegaci n Reactiva: Veh culo de Braitenberg (Caso Base)", fontsize=12, fontweight='bold')
plt.xlabel("Posici n X [m]")
plt.ylabel("Posici n Y [m]")
plt.grid(True, linestyle='--', alpha=0.7)
plt.axis('equal')
plt.legend(loc='upper left')
plt.tight_layout()

# Guardar figura en alta calidad
plt.savefig("braitenberg_base_oficial.png", dpi=300)
plt.show()
```

```
joel@UNAL-Robotica:~/lab5_navegacion$ python3 braitenberg_base.py
================================================================
Iteración  | s_left     | s_right    | v (m/s)    | w (rad/s)
================================================================
0          | 0.116      | 0.108      | -3.131     | -0.136
20         | 0.110      | 0.102      | -2.962     | -0.139
40         | 0.107      | 0.099      | -2.881     | -0.140
60         | 0.107      | 0.099      | -2.876     | -0.140
80         | 0.109      | 0.101      | -2.947     | -0.139
100        | 0.115      | 0.107      | -3.103     | -0.135
120        | 0.124      | 0.116      | -3.369     | -0.130
140        | 0.139      | 0.132      | -3.798     | -0.124
160        | 0.164      | 0.158      | -4.508     | -0.114
180        | 0.211      | 0.205      | -5.823     | -0.102
200        | 0.327      | 0.322      | -9.075     | -0.083
220        | 1.794      | 1.792      | -50.199    | -0.029
221        | 2.797      | 2.796      | -78.291    | -0.017
================================================================
Posición Final Alcanzada: X = 51.36 m, Y = 85.82 m
================================================================
```

Figura 3: Tabla de la evolución progresiva de los sensores y comandos de control.

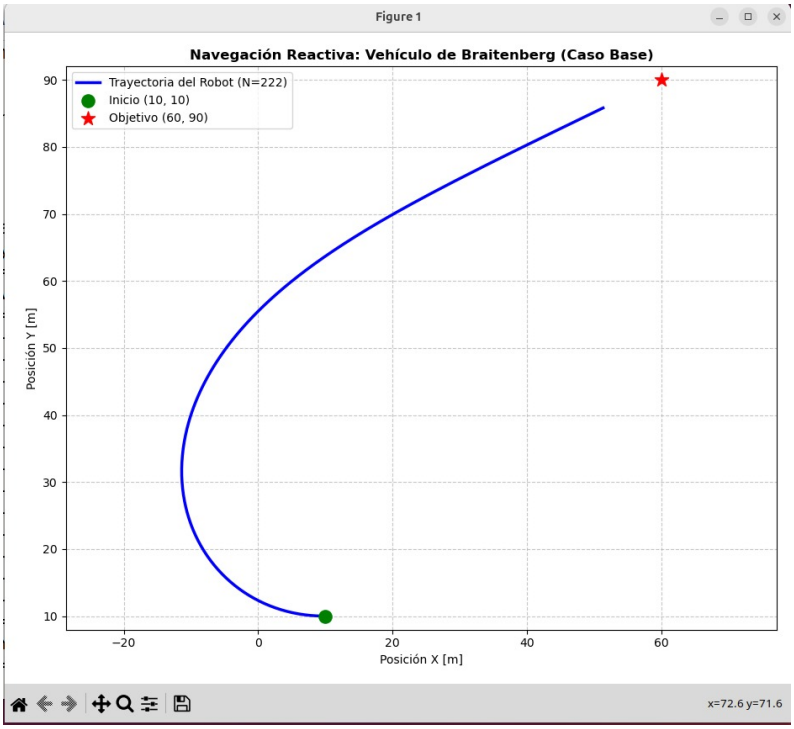


Figura 4: Trayectoria simulación Braitenberg dirigiéndose al objetivo.

La figura muestra la trayectoria del robot en la actividad de navegación reactiva tipo Braitenberg. El robot parte desde la posición $(10, 10)$ y se dirige hacia el objetivo ubicado en $(60, 90)$. La trayectoria obtenida presenta una curvatura progresiva, producto de la diferencia entre las lecturas de los sensores laterales, lo que genera correcciones angulares durante el desplazamiento. Se observa que el robot no alcanza exactamente el objetivo, pero sí logra aproximarse de manera clara, finalizando en la posición $(51{,}36, 85{,}82)$, lo cual evidencia el funcionamiento del control reactivo.

Respuestas a Preguntas de Análisis

1. **¿Cómo influye la diferencia entre sensores en la dirección del robot?**
   La diferencia entre las lecturas del sensor izquierdo y derecho determina la velocidad que

angular del robot. Si un sensor tiene mayor intensidad que el otro entonces, el robot gira hacia el lado donde la intensidad es mas fuerte para corregir su orientación.

En este caso, el sensor izquierdo fue ligeramente mayor que el sensor derecho durante la mayor parte de su recorrido, en consecuencia el robot generó una rotación negativa y siguió una trayectoria curva hasta aproximarse al objetivo.

2. **¿Qué ocurre cuando el robot se aproxima al objetivo?**
   Cuando el robot se aproxima al objetivo la intensidad medida por ambos sensores crece considerablemente. Por lo tanto, la velocidad angular disminuye en magnitud debido a que las lecturas de ambos sensores son similares, en consecuencia el robot realiza menores correcciones de orientación.

   El aumento excesivo de velocidad del robot puede generar una limitación, debido a que puede ocasionar que el robot sobrepase la posición deseada.

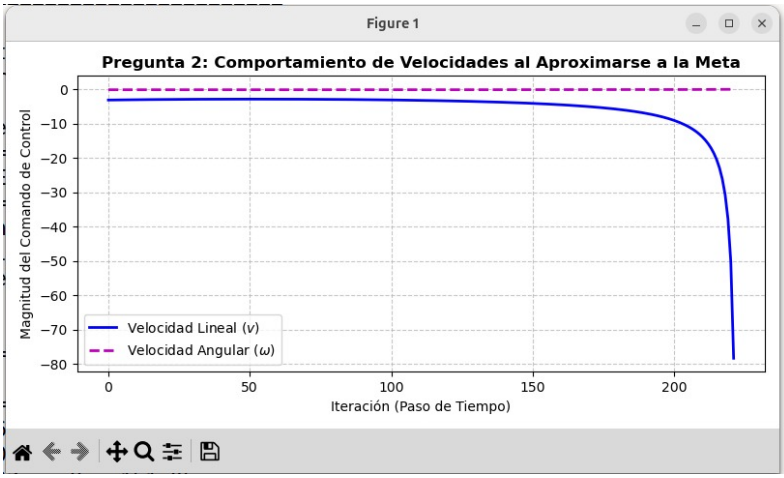


Figura 5

3. **¿Qué limitaciones presenta este enfoque? (¿Qué pasa si se aumenta la cantidad de iteraciones?)**
   Una de las principales limitaciones de la navegación relativa es que el robot solo responde a las lecturas de los sensores. Al incrementar el número de iteraciones de 222 a 600, se observó que el robot siguió desplazándose incluso cuando el robot ya alcanzó su objetivo (178,05, 173,02).

   Esto demuestra que el controlador no dispone de un sistema de parada y aumentar la cantidad de iteraciones no garantiza alcanzar el objetivo con mayor precisión.

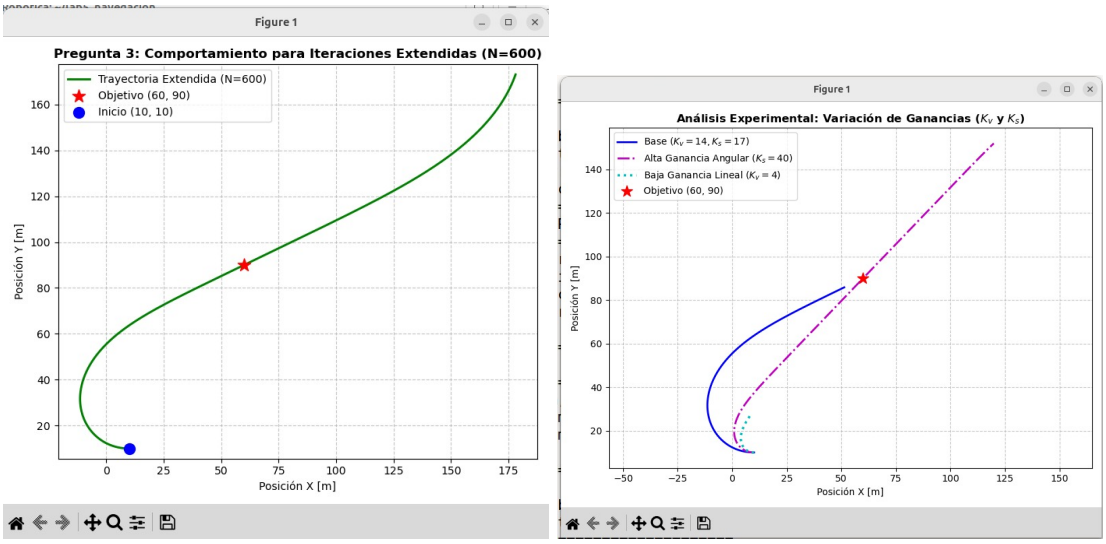


Figura 6

```
======================================================================
RESUMEN DE POSICIONES FINALES ALCANZADAS
======================================================================
1. Caso Base (Kv=14, Ks=17, N=222)   : X = 51.36 m, Y = 85.82 m
2. N Extendidas (Kv=14, Ks=17, N=600): X = 178.05 m, Y = 173.02 m (Orbita la meta)
3. Alta Ks (Kv=14, Ks=40, N=222)     : X = 119.93 m, Y = 151.95 m
4. Baja Kv (Kv=4, Ks=17, N=222)      : X = 8.24 m, Y = 26.97 m (Incompleto)
======================================================================
```

Figura 7

VIDEO: https://youtu.be/-zC6gXDeC58

3. Parte B: Navegación con Algoritmo Bug2

Ejecución de comnados

```
joel@UNAL-Robotica:~/lab5_navegacion/parte_b_bug2$ nano bug2_base.py
joel@UNAL-Robotica:~/lab5_navegacion/parte_b_bug2$ python3 bug2_base.py
```

Figura 8

3.1. Descripción y Código Implementado

El algoritmo Bug2 traza una línea recta teórica (m-line) desde la posición inicial hasta la meta. Si colisiona con un obstáculo, conmuta a modo de seguimiento de contorno hasta cruzarse nuevamente con la m-line a una distancia menor respecto a la meta.

```
    import numpy as np
import matplotlib.pyplot as plt

# ==========================================
# 1. DEFINICI N DEL ENTORNO Y PUNTOS CLAVE
# ==========================================
grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1 # obst culo

```

```
start = np.array([2, 2])
goal = np.array([18, 18])

pos = start.copy()
path = [pos.copy()]

# ==========================================
# 2. FUNCI N DE DETECCI N DE ESPACIO LIBRE
# ==========================================
def is_free(p):
    return grid[p[1], p[0]] == 0

# ==========================================
# 3. CABECERA DE LA TABLA PARA LA TERMINAL
# ==========================================
print("\n" + "="*53)
print(f"| {'Posici n Actual':^15} | {'Siguiente Paso':^15} | {' Libre ?':^11} |")
print("="*53)

# ==========================================
# 4. BUCLE PRINCIPAL DE NAVEGACI N REACTIVA BUG2
# ==========================================
for _ in range(500):
    direction = goal - pos

    # Determina si avanzar primariamente en X o en Y basado en la mayor distancia
    if abs(direction[0]) > abs(direction[1]):
        step = np.array([np.sign(direction[0]), 0])
    else:
        step = np.array([0, np.sign(direction[1])])

    next_pos = pos + step

    # Imprime los datos iterativos en formato de fila de tabla alineada
    libre_str = "S  " if is_free(next_pos) else "No (Obst culo)"
    print(f"| [{pos[0]:2d}, {pos[1]:2d}]          | [{next_pos[0]:2d}, {next_pos[1]:2d}]          | {libre_str:<11} |")

    # L gica simplificada de seguimiento de borde en 4 direcciones ortogonales
    if is_free(next_pos):
        pos = next_pos
    else:
        for move in [np.array([1,0]), np.array([0,1]), np.array([-1,0]), np.array([0,-1])]:
            if is_free(pos + move):
                pos = pos + move
                break

    path.append(pos.copy())

    # Condici n de  xito  al llegar a la meta
    if np.linalg.norm(pos - goal) < 1:
        break

print("-" * 53)
```

```
path = np.array(path)

# ========================================
# 5. VISUALIZACION GR FICA EST TICA
# ========================================
plt.figure(figsize=(8, 8))

# Activamos las propiedades comentadas en la gu a (cmap='gray_r',
    aspect='equal', grid)
plt.imshow(grid, origin='lower', cmap='gray_r')

# Dibujamos la l nea-m directa como referencia visual del algoritmo
    Bug2
plt.plot([start[0], goal[0]], [start[1], goal[1]], color='red',
    linestyle='--', alpha=0.4, label="L nea-m (Directa)")

# Dibujamos la ruta tomada por el robot
plt.plot(path[:,0], path[:,1], 'g-', linewidth=2.5, label="Trayectoria
    Bug2")

# Marcadores destacados para inicio y meta
plt.scatter(start[0], start[1], c='blue', s=120, edgecolors='black',
    zorder=5, label="Inicio")
plt.scatter(goal[0], goal[1], c='red', s=120, edgecolors='black', zorder
    =5, label="Meta")

# Ajustes de est tica general del gr fico
plt.gca().set_aspect('equal')
plt.grid(color='gray', linestyle=':', linewidth=0.5, alpha=0.7)
plt.title("Bug2 Simplificado - Navegaci n Reactiva", fontsize=14,
    fontweight='bold')
plt.legend(loc='lower right', shadow=True, fancybox=True)

plt.show()
```

Listing 1: Simulación simplificada de Bug2

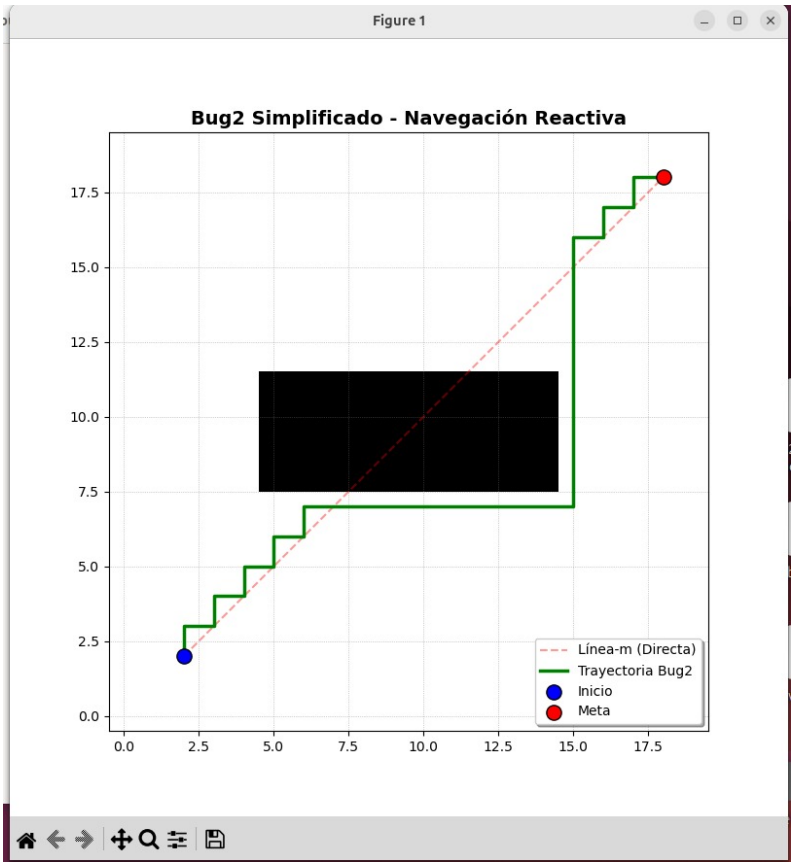


Figura 9: Trayectoria descritas por el robot.

La figura muestra la trayectoria obtenida con el algoritmo Bug2 simplificado sobre un mapa tipo grid. El robot parte desde la coordenada $(2, 2)$ y tiene como objetivo llegar a $(18, 18)$. La línea roja punteada representa la línea-m, es decir, la dirección directa entre el punto inicial y la meta.

Durante el recorrido, el robot detecta un obstáculo rectangular ubicado en la región central del mapa, por lo que interrumpe el avance directo y realiza un bordeo por la parte inferior del obstáculo.

Posteriormente retoma el desplazamiento hacia la meta, alcanzándola finalmente en 32 pasos. El resultado evidencia un comportamiento correcto de evasión de obstáculos y reanudación de la ruta.

VIDEO: https://youtu.be/YZrKQcMLaZY

Paso	Coordenada	Evento Registrado
0	[2, 2]	Inicio de ruta
11	[7, 7]	Obstáculo detectado
19	[15, 7]	Retoma ruta directa
32	[18, 18]	Meta alcanzada

Figura 10: Registro de eventos clave en la trayectoria del robot.

Variando el punto inicial y final

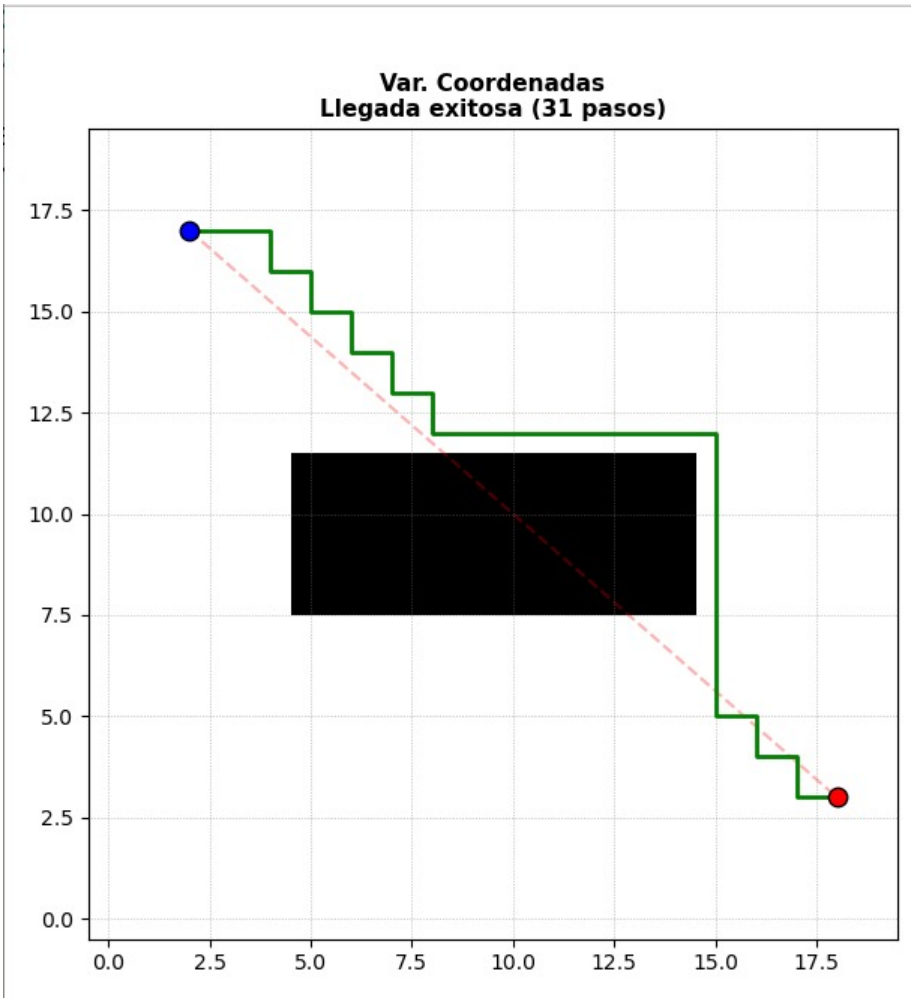


Figura 11

3.2. Respuestas a Preguntas de Análisis (Parte B)

1. **¿El camino obtenido es óptimo? Justifique.**
   El camino obtenido no es el óptimo, porque el algoritmo toma decisiones basándose unicamente en lo que tiene enfrente en ese instante de tiempo.

   Al no analizar el mapa, no encuentra una ruta realmente óptima, por eso si el laberinto o los obstáculos fueran mas encerrados, el robot podría estar dando rodeos innecesarios o caminos muy largo.

2. **¿Qué sucede en entornos complejos?**
   En entornos mas complejos como obstáculos cóncavos, laberintos, el algoritmo puede generar recorridos innecesarios o una navegación menos eficiente. Esto ocurre porque el algoritmo usa reglas simples que se determinan según su entorno en ese instante de tiempo.

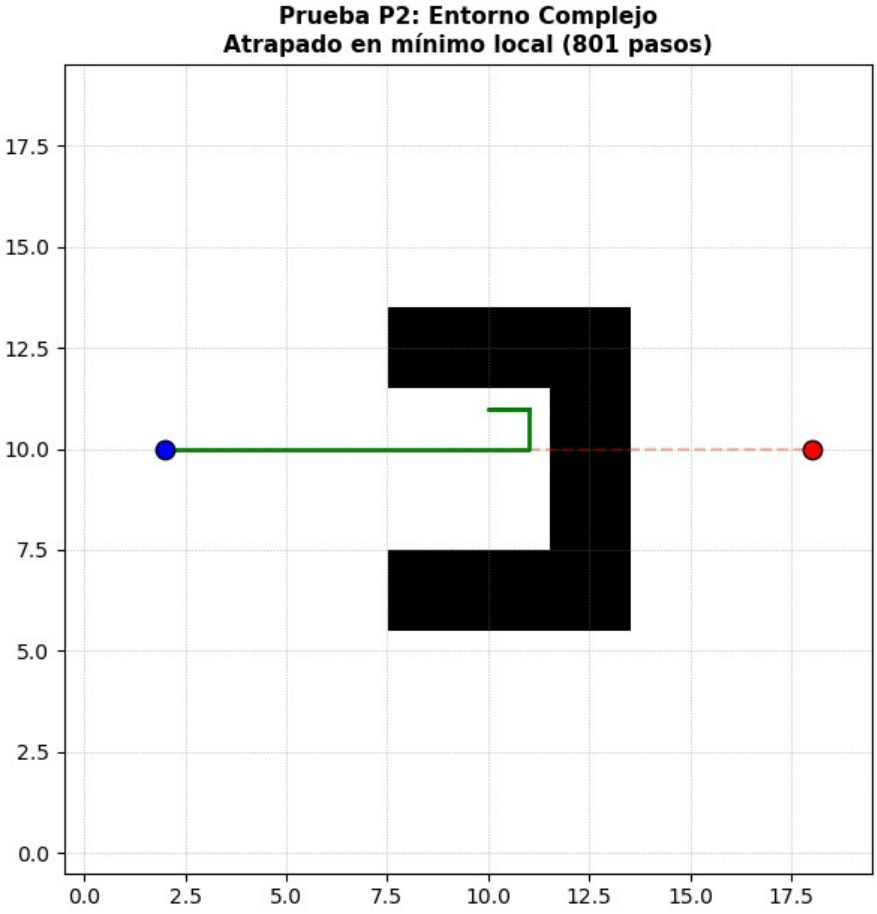


Figura 12

3. **Compare con el enfoque reactivo de la Parte A.**

   El enfoque reactivo de la Parte A se basa únicamente en sensores y genera un movimiento continuo, sin utilizar un mapa del entorno. En cambio, el algoritmo Bug2 simplificado trabaja sobre una grilla y considera la presencia de obstáculos, lo que le permite encontrar rutas viables hacia la meta.
   Por ello, la navegación reactiva es más simple pero menos robusta ante bloqueos, mientras que Bug2 ofrece una estrategia más estructurada para evasión de obstáculos.

4. Parte C: Planificación de Trayectorias con A^*

4.1. Comandos de ejecución

Para iniciar el entorno de trabajo y ejecutar el script base de esta actividad, se utilizaron los siguientes comandos en la terminal de Linux:

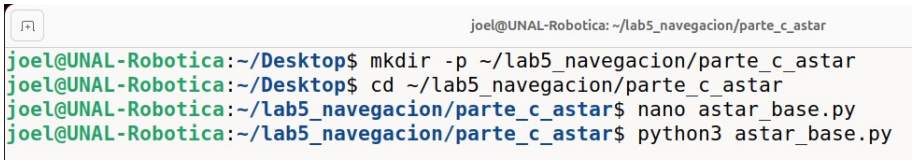


Figura 13: Comandos de inicialización y ejecución en la terminal.

4.2. Descripción y Código Implementado

```python
import heapq
import numpy as np
import matplotlib.pyplot as plt

# ==========================================
# 1. DEFINICI N DEL ENTORNO
# ==========================================
grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1

start = (2, 2)
goal = (18, 18)

# ==========================================
# 2. ALGORITMO A*
# ==========================================
def heuristic(a, b):
    # Distancia Euclidiana
    return np.linalg.norm(np.array(a) - np.array(b))

def astar(grid, start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))

    came_from = {}
    cost = {start: 0}

    # Nodos explorados para an lisis de rendimiento
    nodos_explorados = 0

    while open_list:
        _, current = heapq.heappop(open_list)
        nodos_explorados += 1

        if current == goal:
            break

        # Movimientos ortogonales (4-conectado)
        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            neighbor = (current[0] + dx, current[1] + dy)

            # Verificaci n de l mites y obst culos
            if (0 <= neighbor[0] < grid.shape[0] and
                0 <= neighbor[1] < grid.shape[1] and
                grid[neighbor[1], neighbor[0]] == 0):

                new_cost = cost[current] + 1

```

```
                # Actualizaci n de costos y cola de prioridad
                if neighbor not in cost or new_cost < cost[neighbor]:
                    cost[neighbor] = new_cost
                    priority = new_cost + heuristic(neighbor, goal)
                    heapq.heappush(open_list, (priority, neighbor))
                    came_from[neighbor] = current

    # Reconstrucci n de la ruta
    path = []
    node = goal
    while node != start:
        path.append(node)
        node = came_from.get(node, start) # Evita errores si no hay ruta
    path.append(start)
    path.reverse()

    return path, nodos_explorados, list(cost.keys())

# Ejecuci n
path, nodos_exp, explored_nodes = astar(grid, start, goal)
path = np.array(path)

# ==========================================
# 3. SALIDA POR TERMINAL (TABLA DE RENDIMIENTO)
# ==========================================
print("\n" + "="*50)
print(f"| {'M TRICA DE PLANIFICACI N A*':^46} |")
print("="*50)
print(f"| {'Punto de Inicio':<25} | {str(start):<18} |")
print(f"| {'Punto de Meta':<25} | {str(goal):<18} |")
print(f"| {'Longitud de la Ruta':<25} | {len(path)} celdas{' ':<7} |")
print(f"| {'Nodos Explorados':<25} | {nodos_exp} celdas{' ':<7} |")
print("-" * 50 + "\n")

# ==========================================
# 4. VISUALIZACI N GR FICA
# ==========================================
plt.figure(figsize=(8, 8))
plt.imshow(grid, origin='lower', cmap='gray_r')

# Graficar espacio explorado (opcional, visualmente educativo)
exp_arr = np.array(explored_nodes)
plt.scatter(exp_arr[:,0], exp_arr[:,1], c='lightblue', s=20, alpha=0.3,
    label="Espacio Explorado")

plt.plot(path[:,0], path[:,1], 'r-', linewidth=2.5, label="A* Path")
plt.scatter(start[0], start[1], c='blue', s=120, edgecolors='black',
    zorder=5, label="Inicio")
plt.scatter(goal[0], goal[1], c='red', s=120, edgecolors='black', zorder
    =5, label="Meta")

plt.gca().set_aspect('equal')
plt.grid(color='gray', linestyle=':', linewidth=0.5, alpha=0.7)
plt.title("Planificaci n Global  ptima  con A*", fontsize=14,
    fontweight='bold')
plt.legend(loc='lower right')
plt.show()
```

4.3. Resultados de la Simulación Base

Al ejecutar el algoritmo base, se obtuvieron las siguientes métricas de rendimiento en la terminal:

Cuadro 1: Métricas de Planificación Global (Algoritmo A^*)

Parámetro	**Valor Registrado**
Punto de Inicio	(2, 2)
Punto de Meta	(18, 18)
Longitud de la Ruta	33 celdas
Nodos Explorados	305 celdas

Interpretación: La tabla refleja que el algoritmo A^* logra encontrar una ruta de 33 celdas hacia el objetivo. Para garantizar que esta ruta es efectivamente la óptima, el algoritmo evalúa y explora un total de 305 celdas del entorno.

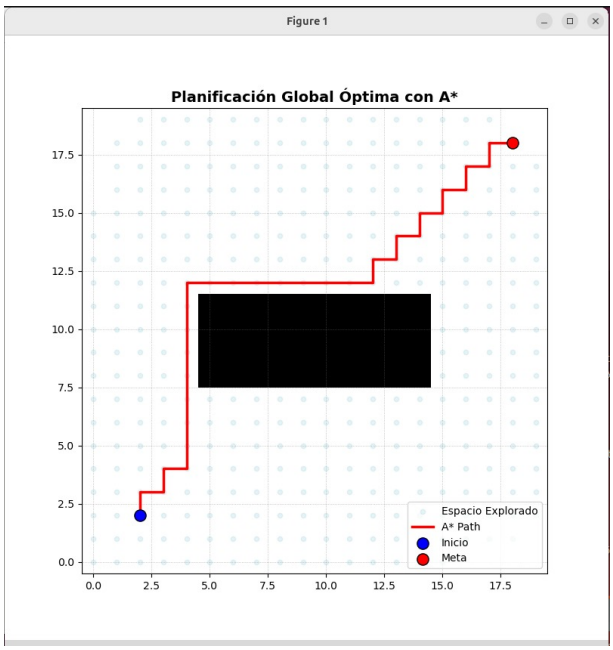


Figura 14: Planificación óptima calculada mediante A^*. Los puntos celestes representan el espacio explorado.

Interpretación: Como se observa en la gráfica, A^* traza una ruta óptima (línea roja) sorteando el obstáculo rectangular. A diferencia de una búsqueda ciega, el espacio explorado (puntos celestes) no se expande uniformemente por todo el mapa, sino que se mantiene contenido y dirigido directamente hacia la meta superior derecha, evidenciando la eficiencia computacional de la heurística utilizada.

4.4. Cambiando los parámetros

Para evaluar la eficiencia del algoritmo, se realizó un cambio en las coordenadas de inicio/meta y se agregó un obstáculo en forma de U para crear un laberinto con un mínimo local.

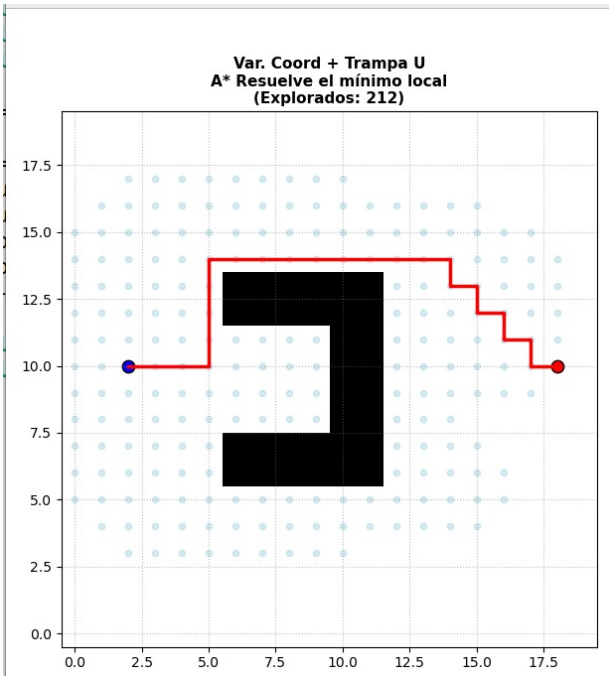


Figura 15: Variación de coordenadas y trampa en U. Se exploraron 212 nodos en total.

Interpretación: Se demuestra que A^* encuentra la ruta óptima sin quedarse atrapado. A diferencia de los algoritmos puramente reactivos, A^* planifica una ruta alternativa rodeando el obstáculo por el exterior de manera global y eficiente.

4.5. Respuestas a Preguntas de Análisis (Parte C)

1. **¿Qué diferencia hay entre BFS, UCS y A^*?**
   La principal diferencia entre estos tres algoritmos está en la manera y dirección en que buscan una ruta hacia el objetivo:

   - **BFS (Búsqueda en Anchura):** Realiza una búsqueda por niveles en todas las direcciones, explorando primero los nodos más cercanos al punto inicial en número de movimientos. Es útil solo cuando todos los movimientos tienen el mismo costo.
   - **UCS (Búsqueda de Costo Uniforme):** Considera el costo real de cada movimiento acumulado desde el inicio. Encuentra la ruta de menor costo total, pero su expansión radial no tiene un sentido de dirección.
   - **A^*:** Combina el principio de UCS con una función heurística. Considera tanto el costo acumulado como una estimación de la distancia faltante. Esto le otorga "direccionalidad", orientando la búsqueda y reduciendo drásticamente la cantidad de nodos explorados.

2. **¿Cuál es el papel de la heurística?**
   La heurística $h(n)$ cumple la función de estimar el costo o la distancia que falta desde un nodo hasta el objetivo, permitiendo que A^* seleccione las posiciones más prometedoras para expandirse. Funciona como un "sentido de orientación".

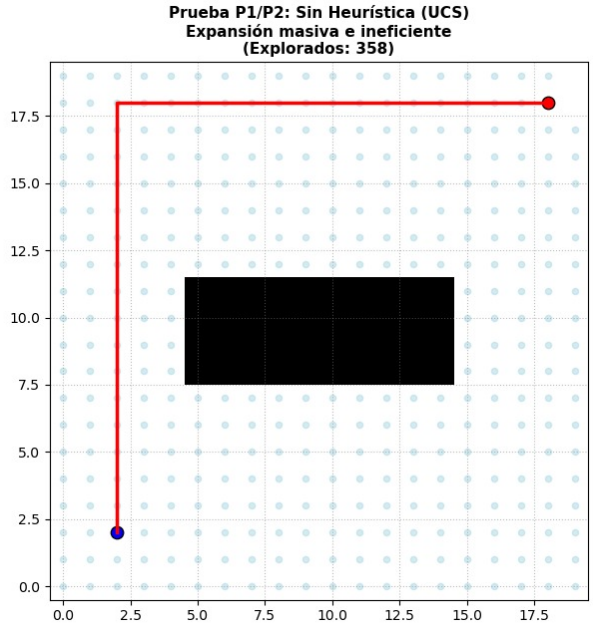


Figura 16: Prueba P1/P2: A^* sin heurística ($h = 0$). Expansión masiva con 358 nodos explorados.

Interpretación: Si se anula la heurística ($h = 0$), el algoritmo se comporta exactamente como UCS. Como evidencia la gráfica, al carecer de heurística se produce una expansión masiva (burbuja gigante) en todas direcciones. Esto demuestra que el papel vital de la heurística es evitar este desperdicio de recursos computacionales.

3. **¿Qué ocurre si la heurística no es adecuada?**
   Si la heurística no es adecuada, se rompe el equilibrio entre optimalidad y velocidad del algoritmo.

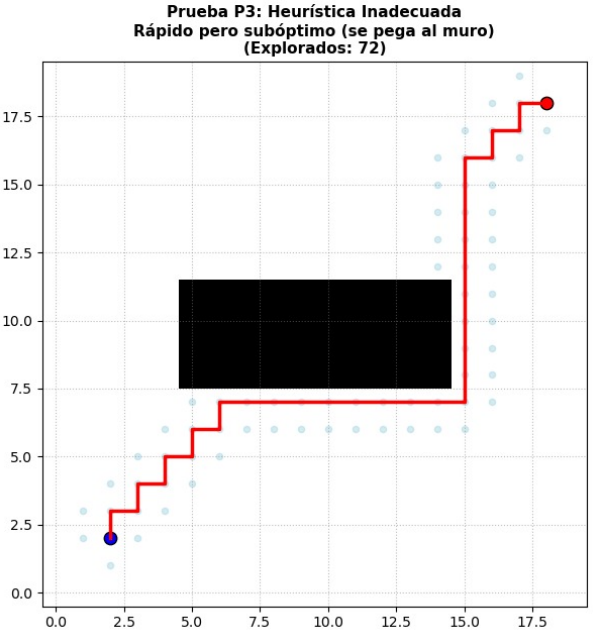


Figura 17: Prueba P3: A^* con heurística sobrestimada. Alta velocidad (72 nodos) pero trayectoria subóptima.

Interpretación: Si se utiliza un A^* con una heurística **inadecuada** (sobrestimada), el

comportamiento cambia drásticamente. Se muestra cómo el algoritmo explora muy pocos nodos (es extremadamente rápido computacionalmente), pero corre el riesgo de trazar una ruta subóptima al volverse demasiado agresivo, lo cual se evidencia al ver cómo la trayectoria roja se pega a los muros de manera ineficiente.

5. Parte D: Planificación con Mapas de Ocupación y Transformada de Distancia

5.1. Comandos de ejecución

Para iniciar el entorno de trabajo y ejecutar el script base correspondiente a la transformada de distancia, se utilizaron los siguientes comandos en la terminal de Linux:

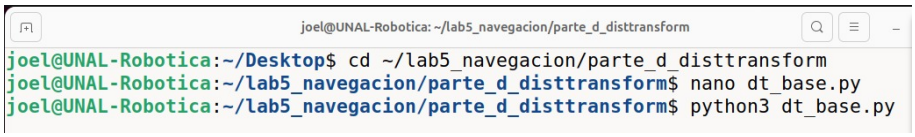


Figura 18: Comandos de inicialización y ejecución para la Parte D.

5.2. Descripción y Código Implementado

El método de Transformada de Distancia (Distance Transform) es un planificador global que inunda el mapa partiendo desde la meta hacia el exterior. A diferencia de A^*, que busca la meta desde el inicio, este algoritmo genera un campo escalar o "mapa de calor"donde el valor de cada celda representa su distancia libre de colisiones hasta el objetivo. Posteriormente, el robot solo debe descender por el gradiente.

```
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# MAPA
# ========================================
grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1 # obst culo

goal = (2, 2)

# ========================================
# DISTANCE TRANSFORM (NO  PTIMO )
# ========================================
dist = np.ones_like(grid) * np.inf
dist[goal[1], goal[0]] = 0

print("\nCalculando propagaci n de distancias...")
for iteracion in range(100):
    for x in range(20):
        for y in range(20):
            if grid[y, x] == 0:
                neighbors = []
                for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < 20 and 0 <= ny < 20:
                        neighbors.append(dist[ny, nx])
                dist[y, x] = min(dist[y, x], min(neighbors) + 1)
```

```python

# ========================================
# SEGUIMIENTO DE CAMINO
# ========================================
start = (18, 18)
pos = start
path = [pos]

print("\nTrazando ruta por descenso de gradiente...")
for _ in range(200):
    x, y = pos
    neighbors = []
    for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 20 and 0 <= ny < 20:
            neighbors.append(((nx, ny), dist[ny, nx]))

    pos = min(neighbors, key=lambda item: item[1])[0]
    path.append(pos)

    if pos == goal:
        break

path = np.array(path)
```

5.3. Resultados de la Simulación Base

Al ejecutar el algoritmo, la terminal arrojó los siguientes resultados sobre el trazado de la ruta por descenso de gradiente:

Cuadro 2: Resultados de Propagación - Distance Transform

Parámetro	**Valor Registrado**
Punto de Inicio	(18, 18)
Punto de Meta (Origen del campo)	(2, 2)
Pasos de la ruta (Distancia Manhattan)	33

Interpretación: Como se detalla en la tabla, el campo de distancias se calculó tomando como sumidero (origen de la propagación) la coordenada (2, 2). El robot, ubicado inicialmente en (18, 18), logró descender el gradiente hasta la meta en exactamente 33 pasos, lo cual corresponde a la distancia Manhattan mínima esperada.

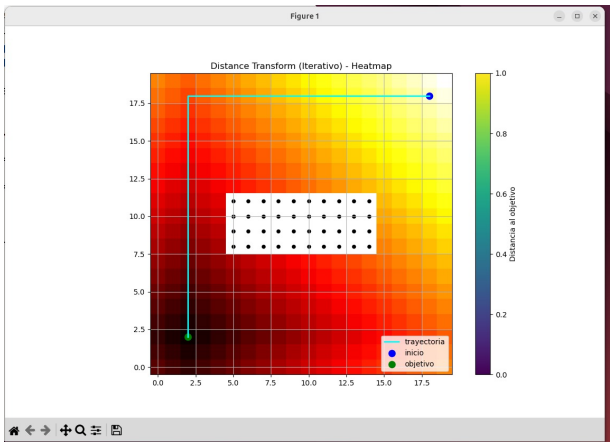


Figura 19: Mapa de calor y trayectoria reconstruida mediante descenso de gradiente.

Interpretación: En la gráfica de calor obtenida, los colores cálidos (blanco/amarillo) representan zonas de mayor distancia a la meta, mientras que los colores oscuros representan proximidad. La trayectoria adopta una forma de "L" escalonada desplazándose horizontalmente a la izquierda y luego verticalmente hacia abajo. Este comportamiento no es un error, sino una característica matemática de la conectividad de 4 vecinos: en geometría Manhattan, avanzar primero en el eje X y luego en el Y resulta en el mismo costo óptimo (33 pasos) que avanzar en zigzag.

5.4. Cambiando los parámetros

Para comprobar la robustez del algoritmo, se modificó el mapa de ocupación añadiendo obstáculos aleatorios que forman un laberinto, y se invirtieron las posiciones de análisis: Inicio en (2, 18) y Meta en (18, 2).

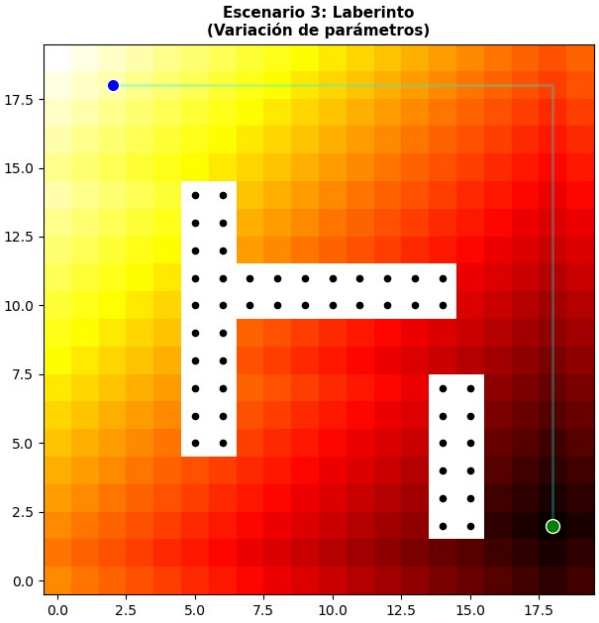


Figura 20: Modificación del entorno (laberinto) y coordenadas.

Interpretación: La Figura 20 demuestra que el algoritmo generaliza correctamente ante la variación de parámetros. Al inundar el mapa con el cálculo de distancias, el robot (punto azul) es

capaz de fluir a través del pasillo libre de obstáculos hasta encontrar el sumidero (punto verde) en la parte inferior derecha, confirmando su capacidad para resolver laberintos.

5.5. Respuestas a Preguntas

1. **¿Por qué este método garantiza optimalidad?**
   Garantiza optimalidad porque genera una función de navegación global libre de mínimos locales.

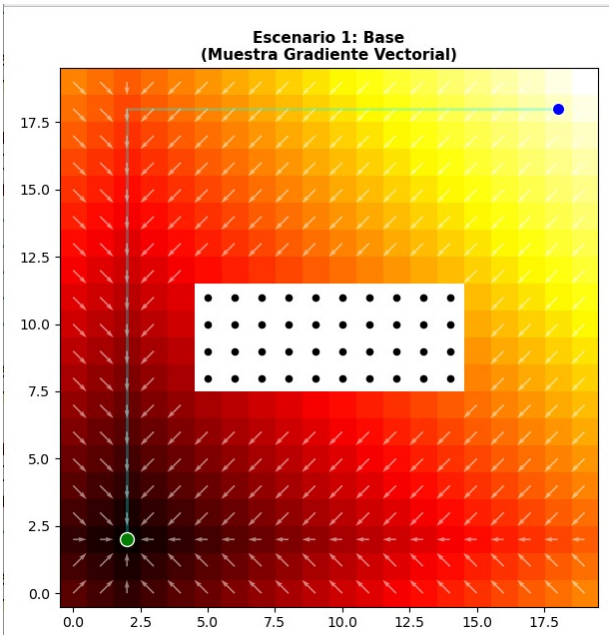


Figura 21: Gradiente vectorial extraído del mapa de distancias.

Interpretación: Como se ilustra en la Figura 21, al derivar el mapa de distancias se obtiene un campo vectorial continuo. En cualquier celda libre del mapa, las flechas (vectores del gradiente) apuntan incondicionalmente hacia la ruta más corta al sumidero global (la meta en 2,2). Al seguir estas flechas, es matemáticamente imposible que el robot se desvíe o quede atascado.

2. **¿Cuál es su costo computacional?**
   El costo computacional es muy elevado (fuerza bruta). Su complejidad está estrictamente ligada al área total del espacio de trabajo $\mathcal{O}(N \times M \times iteraciones)$. En esta simulación de 20×20, el programa realiza aproximadamente 40,000 revisiones de celdas para converger. Requiere barrer y actualizar iterativamente toda la matriz de ocupación múltiples veces para propagar la "onda"de distancias, independientemente de qué tan cerca o lejos esté el robot, haciéndolo ineficiente para mapas a gran escala en comparación con algoritmos heurísticos como A^*.

3. **Compare con el algoritmo Bug2.**
   La diferencia fundamental entre ambos radica en el nivel de conocimiento e información espacial del entorno:

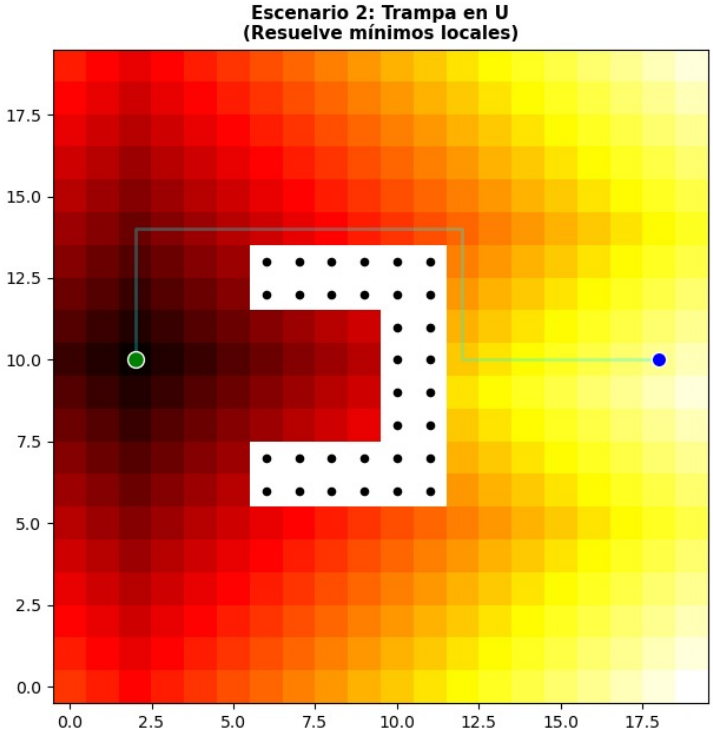


Figura 22: Escenario 2: Resolución de trampa cóncava en Ü".

Interpretación:

- **Bug2 (Reactivo/Local):** Funciona a ciegas. Como se demostró en la Parte B, al entrar en un obstáculo cóncavo (trampa en "U"), Bug2 queda atrapado en un bucle infinito al no tener memoria global del mapa.
- **Distance Transform (Global):** Al inundar todo el espacio previamente, el algoritmo ya conoce la geometría cóncava de la trampa. Como se observa en la Figura 22, el gradiente de colores fluye naturalmente rodeando el muro externo de la "U". El robot, sin necesidad de entrar a ciegas en la trampa, sigue la ruta segura esquivando el obstáculo desde afuera y garantizando la llegada al objetivo.